URP Migrator: Built-in to URP Shader and Effect Converter

Unity is retiring the Built-in Render Pipeline: it is deprecated in Unity 6.5, still supported in 6.7 LTS until the end of 2028, and gone in Unity 7. Unity's Render Pipeline Converter upgrades Unity's own materials, but custom shaders, surface shaders, GrabPass and `OnRenderImage` image effects have to be rewritten by hand. URP Migrator does that rewrite for you.

- **Scans** your project for everything that breaks or changes in URP and explains what to do, with HTML, Markdown and CSV reports.
- **Converts surface shaders** to URP shaders with Forward, Shadow, Depth, Depth Normals and Meta passes. Your `surf`, vertex, `finalcolor` and custom lighting functions are kept as they are.
- **Converts lit vertex/fragment shaders** (ForwardBase/ForwardAdd, Built-in shadow macros, GrabPass).
- **Converts `OnRenderImage` image effects** so they run in URP through a renderer feature.
- **Safe:** preview every change, originals are backed up automatically, and one click restores them.
- **Keeps material property names**, so existing materials, prefabs and scripts keep working.
- Converted shaders are self-contained: they do not need URP Migrator after conversion.

---

1. Requirements

Item	Requirement
Unity	Unity 6 (6000.0) or later. Tested on 6000.0.84f1 on Apple Silicon.
URP	Universal RP 17.0 or later for converting. Scanning works without URP.
Render paths	Converted shaders were tested in URP Forward, Forward+ and Deferred.
Graphics APIs	Converted shaders compile for Direct3D 11, Metal, Vulkan and OpenGL ES 3.

2. Quick start (5 minutes)

1. Back up or commit your project (URP Migrator also backs up every file it changes).
2. Install **Universal RP** from the Package Manager if it is not installed yet.
3. Open **Window ▸ Rendering ▸ URP Migrator** and click **Scan Project**.

4. Review the list. Rows marked **Auto** are converted automatically; **Partial** converts with notes; **Manual** explains what to change by hand. Click **Preview Conversion** to see the exact changes.
5. Click **Convert Checked**. URP Migrator backs up the originals, writes the URP versions, switches on the URP settings the converted content needs, and compiles every converted shader.
6. Convert Unity's own Built-in materials (Standard, Legacy, Mobile, Particles) with **Unity Material Converter** in the toolbar, which opens Unity's Render Pipeline Converter.
7. Not happy? Open the **Backups** tab and click **Revert**.

Try it first on the demo: open `URPMigrator/Demo/URP Migrator Demo.unity`. In a URP project the demo objects render pink. Scan, convert, and they render as they did in the Built-in pipeline.

3. The URP Migrator window

Findings lists one row per problem:

- **Impact**: *Breaks* (renders pink, renders nothing, or the code stops working in URP), *Works differently*, or *Info*.
- **Type**: surface shader, lit vertex/fragment shader, GrabPass shader, image effect, camera command buffer, Projector, Post Processing v2, Shader.Find, project setting, and others.
- **Fix**: *Auto*, *Partial* (converted, but read the notes), *Manual*, or *Unity tool* (Built-in materials, handled by Unity's converter).
- **Used by**: how many materials (or scenes and prefabs) use the asset.

Select a row to see what URP does differently, what to do, the Built-in features found in the file, conversion notes, and where the asset is used. Double-click a row to open the file.

Toolbar:

- **Scan Project**: scans the whole Assets folder, or the folder chosen under **Scope**.
- **Convert Checked**: converts the ticked rows.
- **Export Report**: writes HTML (for sharing), Markdown (for tickets and pull requests) or CSV (for spreadsheets).
- **Unity Material Converter**: opens Unity's Render Pipeline Converter for Built-in materials.

Backups lists every conversion with its files. **Revert** restores them exactly; if a file changed again after the conversion, you are told before it is overwritten.

Options:

- **Keep the Built-in SubShader (dual pipeline):** adds the URP SubShader in front of the original, so the same shader works in both pipelines. Use it for assets you sell or share to both pipelines.

- **Run vertex modifiers in shadow and depth passes:** Built-in only did this with `addshadow`. Turn it on so displaced geometry casts matching shadows.
- **Switch on the URP settings converted content needs** (on by default).
- **Compile converted shaders and report errors** (on by default).

You can also right-click a shader or script in the Project window and choose **URP Migrator ▸ Preview Conversion**.

4. What gets converted

4.1 Surface shaders

URP has no surface shaders. URP Migrator generates a URP SubShader that calls your original functions from new passes:

Pass	LightMode	Purpose
ForwardLit	UniversalForwardOnly	Lighting in Forward, Forward+ and Deferred
ShadowCaster	ShadowCaster	Shadows (with URP's shadow bias)
DepthOnly	DepthOnly	Depth prepass and depth texture
DepthNormalsOnly	DepthNormalsOnly	SSAO and other effects that need normals
Meta	Meta	Lightmap baking (albedo and emission)

Lighting models:

- `Standard` and `StandardSpecular` → URP's PBR lighting (the same BRDF as URP/Lit).
- `Lambert` and `BlinnPhong` → URP's simple lighting, with `Specular`, `Gloss` and `_SpecColor` mapped so highlights keep their size.
- **Custom lighting functions** (`LightingX(SurfaceOutput s, half3 lightDir, half atten)`, the `viewDir` variant, and the `UnityGI` variants with `LightingX_GI`) are called once per light: the main light, then every additional light. `_LightColor0` and `_WorldSpaceLightPos0` describe the light being shaded, as in Built-in.

Supported `#pragma surface` options: `alpha`, `alpha:fade`, `alpha:blend`, `alpha:premul`, `alphatest:<var>`, `keepalpha`, `decal:add`, `decal:blend`, `vertex:<func>`, `finalcolor:<func>`, `addshadow`, `fullforwardshadows`, `noshadow`, `noambient`, `nofog`, `nolightmap`, `nodynlightmap`, `nometa`, `noinstancing`, `dithercrossfade`. Forward-path tuning options (`noforwardadd`, `halfasview`, `interpolateview`, ...) are accepted and ignored.

Supported `Input` members: `uv_Tex`, `uv2_Tex`, `uv3_Tex`, `uv4_Tex`, `worldPos`, `worldNormal`, `viewDir` (world or tangent space, as in Built-in), `worldRefl`, `screenPos`, `COLOR`, `VFACE`, `INTERNAL_DATA` with `WorldNormalVector` and `WorldReflectionVector`, and your own members written by a `vertex:` function.

Shadows follow Built-in rules: a shader casts shadows if it had `addshadow` or a Fallback with a shadow pass. `clip()` and `alphatest` cut holes in shadows and depth, as with `addshadow`.

4.2 Vertex/fragment shaders

- `CGPROGRAM` becomes `HLSLPROGRAM` on top of a compatibility layer that provides the `UnityCG.cginc`, `Lighting.cginc`, `AutoLight.cginc` and `UnityPBSLighting.cginc` functions and macros your code uses (`UnityObjectToClipPos`, `ShadeSH9`, `SHADOW_COORDS` / `SHADOW_ATTENUATION`, `UNITY_LIGHT_ATTENUATION`, fog macros, texture macros and more).
- `ForwardBase` passes become `UniversalForwardOnly` with URP's shadow keywords.
- A `ForwardAdd` pass is merged into the base pass and runs once per URP additional light (as Built-in did with one pass per light). It needs the same vertex output as the base pass; otherwise it is dropped and reported.
- `GrabPass` reads URP's **Opaque Texture** (`_CameraOpaqueTexture`). Named grab textures are renamed for you.
- Shadow, depth and depth-normals passes are generated when Built-in took them from a `Fallback` or `UsePass`. Custom `ShadowCaster` passes are kept and also reused as `DepthOnly`.
- Unlit vertex/fragment shaders already render in URP; they are listed as *Info* and left alone.

All material properties move into a `UnityPerMaterial` constant buffer, so converted shaders are **SRP Batcher compatible**.

4.3 Image effects (`OnRenderImage`)

URP never calls `OnRenderImage`. URP Migrator adds an `OnRenderImageURP` method next to it. `Graphics.Blit`, `RenderTexture.GetTemporary` / `ReleaseTemporary`, `Shader.SetGlobal*`, `Graphics.SetRenderTarget` and `GL.Clear` are redirected to URP's command buffer, and helper methods that blit get URP versions too. The class is marked with `IURPImageEffect`.

The **Legacy Image Effects** renderer feature (added to your URP renderers automatically) runs these components on the camera in component order. `[ImageEffectOpaque]` effects run before transparent objects; `[ImageEffectAllowedInSceneView]` effects also run in the Scene view; `depthTextureMode` requests turn on URP's depth or normals textures. The original `OnRenderImage` stays, so the script still works in the Built-in pipeline.

Immediate-mode calls such as `GL.Begin` or `Graphics.DrawMeshNow` cannot be recorded into a command buffer. They are reported so you can move them to `urpmContext.commandBuffer`.

4.4 Project settings

When converted content needs them, URP Migrator turns on the **Opaque Texture** (GrabPass), the **Depth Texture**, and sets **Depth Texture Mode** to *After Opaques* (otherwise transparent shaders that read depth, such as water edges and soft particles, see the previous frame). It also adds the **Legacy Image Effects** and, for shaders with more than two unlit passes, **Render Objects** renderer features. URP assets are backed up first.

5. What is reported but not converted

Found	What to do
Camera or Light `AddCommandBuffer`	Move the commands into a `ScriptableRendererFeature`
`OnPreRender`, `OnPostRender`, `OnPreCull`	Use `RenderPipelineManager.beginCameraRendering` / `endCameraRendering`
`SetReplacementShader`, `RenderWithShader`	Use a Render Objects renderer feature with an override material
Post Processing Stack v2	Recreate the effects as URP Volume overrides
Projector component	Use a URP Decal Projector
Halo, Lens Flare, Flare Layer	Use Lens Flare (SRP)
`Shader.Find("Standard")` and other Built-in shaders	Use `Universal Render Pipeline/Lit`, Simple Lit, Unlit or Particles
Surface shader tessellation (`tessellate:`)	Converted without tessellation (reported)
Fixed-function passes with `Lighting On`, legacy Vertex/Deferred light modes	Rewrite as vertex/fragment passes

6. Accuracy: measured results

URP Migrator was tested on a corpus of 52 Built-in shaders and image effects written for this purpose: 36 surface shaders covering every lighting model, option and `Input` member above, 11 vertex/fragment shaders (ForwardBase, ForwardAdd, GrabPass, depth texture, outlines, custom shadow casters) and 5 image effects.

- **44 of 44** converted shaders compile for Direct3D 11, Metal, Vulkan and OpenGL ES 3, across the URP keyword combinations that matter (shadows, additional lights, Forward+, light layers, SSAO, decals, lightmaps, fog, instancing).
- Each shader was rendered with the Built-in pipeline (original) and with URP (converted) in the same scene. **41 of 44** match within an average difference of 2/255 per pixel and with under 5% of pixels off by more than 16/255. The median difference is **0.46/255**. Unity's own conversion of the Standard shader to URP/Lit, measured the same way, differs by 0.77/255. The results are the same in URP Forward, Forward+ and Deferred.
- The three that differ more: tessellation (not available in URP, reported), a double-sided transparent PBR shader (URP's BRDF treats back faces differently), and a dissolve shader whose holes show back faces (shadowing differs between the pipelines).
- **4 of 4** `OnRenderImage` effects (single blit, multi-pass blur with temporaries, depth fog with `[ImageEffectOpaque]`, vignette) produce the same image in URP: average difference 0.32 to 0.68/255.
- With point and spot lights, every shader (including Unity's own Standard → URP/Lit) looks different from Built-in because URP uses a different light falloff. Converted shaders differ by the same amount as Unity's conversion.

7. Command line and CI

```
Unity -batchmode -projectPath <project> -executeMethod CedarLabs.URPMigrator.MigratorCL
      [-urpmFolder Assets/Art] [-urpmReport Reports/urp.html] [-urpmReport Reports/urp.
      [-urpmConvert] [-urpmDual] [-urpmNoSettings] [-urpmNoValidate] [-urpmFailOnBreaks

Unity -batchmode -projectPath <project> -executeMethod CedarLabs.URPMigrator.MigratorCL
```

Exit codes: `0` success, `1` items that break in URP remain (with `-urpmFailOnBreaks`), `2` errors or converted shaders that do not compile.

8. Safety

- Nothing changes until you click **Convert**.
- Before writing, every file is copied to `URPMigrator_Backups/<date-time>/` in your project folder (outside Assets, so Unity does not import it). **Revert** restores it exactly.
- Converted shaders keep their names, GUIDs and property names: materials, prefabs, scenes and scripts that reference them keep working.

- Converted shaders contain the compatibility code they need. You can remove URP Migrator afterwards. Converted image effects need the small runtime folder (`URPMigrator/Runtime`).

9. Known limitations

- Standard lighting uses URP's BRDF, so highlights can differ slightly from Built-in Standard, as with Unity's own converter.
- Additional lights use URP's falloff and URP's per-object light limit.
- Built-in vertex lights (`Shade4PointLights`) are approximated with URP's additional lights.
- Shaders that read `_CameraDepthTexture` or the grab texture through `sampler2D` are not ready for single-pass instanced XR.
- Adaptive Probe Volumes and DOTS instancing (GPU Resident Drawer batching) are not used by converted shaders; they use light probes and regular instancing.
- `_LightTexture0` , `_LightMatrix0` and other Built-in light-cookie internals have no URP value (reported).
- Tested on Unity 6000.0 with URP 17.0. URP keywords are read from your installed URP version, so newer Unity 6 releases get matching variants.

10. FAQ

Will my materials break? No. The converted shader keeps the same file, name and property names.

Can I keep supporting the Built-in pipeline? Yes: turn on *Keep the Built-in SubShader*. The shader then works in both pipelines.

What about Shader Graph? Shader Graph shaders already support URP and are skipped.

Does it change my scenes? No. It only edits the shader and script files you convert, and the URP pipeline assets.

Can I convert one file to check the result? Yes: select it and click **Convert This**, or use **Preview Conversion** first.

11. Support

Use the support email on URP Migrator's Asset Store page. Include your Unity and URP versions and the exported Markdown report (**Export Report ▸ Markdown**). A response usually arrives within 2 business days.

URP Migrator is made by Cedar Labs.